Java Programming Introduction

- Dr.R.Anusuya



Java

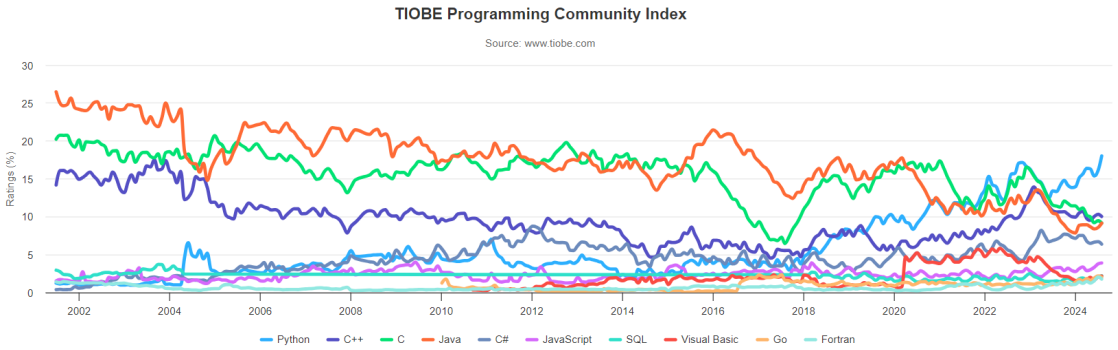
Java is a software platform and programming language that is class based, object-oriented and intended for developers to create applications.

Object Oriented Programming Model

- Java is object oriented programming model.
- Everything is considered as object in Java.
- An object always would have a state and a behavior.

Applications of Java

Popularity of Java as on Aug 2024



Creation of Java

- The creation of C(1989) is considered by many to have marked the beginning of the modern age of computer languages.

As the complexity of programs increased beyond 250000 lines of code C could not handle and hence C++ was developed as an object oriented programming language.

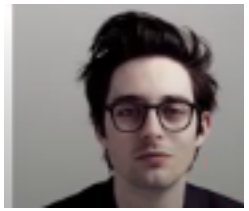
- C++ also called C with classes and was widely used in early 1990s.
- Within few years internet and www was introduced which called for a new programming revolution.

History of Java

- The history of Java starts with Green Team of Sun Microsystems.
- The team comprises of
- James Gosling
- Mike Sheridan
- Patrick Naughton







Originally named as Greentalk

- The team initiated the Java language project in 1991.
- Team initiated a task to develop a language to be used in digital devices such as televisions and set-top boxes.
- It was named “Greentalk” and file extension was .gt.

Renamed to Oak

- Later it was renamed to “Oak”.
- Name is inspired from “Oak Tree” which was outside Gosling’s office.

Why Oak? Oak is a symbol of strength and chosen as a national tree of many countries like the U.S.A., France, Germany, Romania, etc.

- In 1995, the team decided to rename because it was already a trademark by Oak Technologies.



New Name was Java

- The team gathered to choose a new name.
- The suggested words were "dynamic", "revolutionary", "Silk", "jolt", "DNA", etc.

They wanted something that reflected the essence of the technology: revolutionary, dynamic, lively, cool, unique, and easy to spell and fun to say.

According to James Gosling, "Java was one of the top choices along with Silk". Since Java was so unique, most of the team members preferred Java than other names.



Java

Java is an island of Indonesia where the first coffee was produced (called java coffee). It is a kind of espresso bean. Java name was chosen by James Gosling while having coffee near his office.

Official Release

- Java was selected as one of the ten best products of 1995 by Time Magazine
- JDK 1.0 released in(January 23, 1996).
- After the first release of Java, there have been many additional features added to the language.

Now Java is being used in Windows applications, Web applications, enterprise applications, mobile applications, cards, etc. Each new version adds the new features in Java.

JDK VERSION RELEASED SO FAR

- Many java versions have been released till now. The current stable release of Java is Java SE 22.0.2.
- JDK Alpha and Beta (1995) 13. Java SE 11 (September, 25th 2018)
- JDK 1.0 (23rd Jan 1996) 14. Java SE 12 (March, 19th 2019)
- JDK 1.1 (19th Feb 1997) 15. Java SE 13 (September, 17th 2019)
- J2SE 1.2 (8th Dec 1998) 16. Java SE 14 (March 17,2020)
- J2SE 1.3 (8th May 2000) 17. Java SE 15 (September 15,2020)
- J2SE 1.4 (6th Feb 2002) 18. Java SE 16 (March 16,2021)
- J2SE 5.0 (30th Sep 2004) 19. Java SE 17 (September 2021)
- Java SE 6 (11th Dec 2006) 20. Java SE 18 (March, 22nd 2022)
- Java SE 7 (28th July 2011) 21. Java SE 19 (September, 20th 2022)
- Java SE 8 (18th Mar 2014) 22. Java SE 20 (March 21,2023)
- Java SE 9 (21st Sep 2017) 23. Java SE 21 (September 2023)
- Java SE 10 (20th Mar 2018) 24. Java SE 22 (March 19,2024)
- 25. Java SE 23 (September 17.2024)
- 26. Java SE 24 (March 18, 2025)
- 27. Java SE 25 (September 16,2025)
- 28. Java SE 26 (March 17, 2026)

Java as Open Source

On November 13, 2006, Sun Microsystems released Java as free and open source software under the terms of General Public License(GPL).

- Oracle corporation acquired Java in 2009-2010.

Java Buzzwords

Java Buzzwords

- Simple – Easy to learn and use effectively.
- Object oriented – clean, usable, pragmatic approach to objects.
- Robust – Checks code at compile and run time. Handles memory management and exceptional conditions effectively.
- Multithreaded – supports multithreaded programming which helps in doing many things simultaneously.

Java Buzzwords

Architectural neutral- Java Virtual Machine(JVM) helps to achieve “Write Once Run anyhwhere” regardless of OS, processor and core system.

- Interpreted and High Performance – Java was designed to perform well on very low power CPUs.
- Distributed – Java is designed for the distributed environment of the internet
- Dynamic – can accesses object at run time.

Security- Java program runs only in the java execution environment and not allowing to access other parts of the computer.

- Portable – Only JVM needs to be implemented for each platform.

Why Java is important for internet

- Internet requires dynamic, portable and secure programming language for transmission of contents over the internet.
- Java provides these features and hence became popular for internet applications.

Reason to be popular

- Java has worldwide popularity.
- Platform independence.
- Parallel and distributed environment.
- Base for Android mobile platform.
- Java offers backward compatibility.
- Reliability, Speed and Performance.

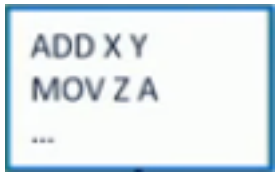
Java Programming Features

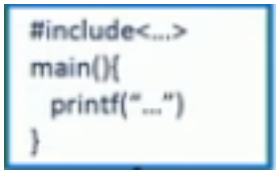
Java Programming Features

- Java core –system software or application software.
- Java Applet – for developing GUIs.
- Java Internet – for implementing network programming

Programming Langauage

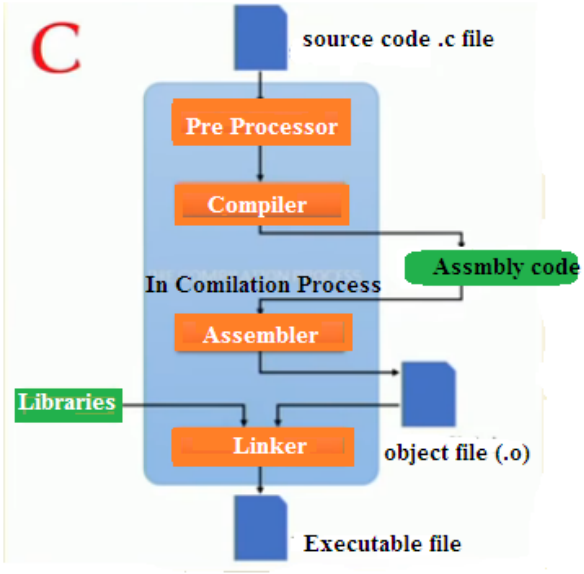
- Machine Level – in 0’s and 1’s
- Assembly Level –
- High Level –
- Every program is converted to machine level to be understood by the system.
- High level language is converted to machine level with the help of interpreter and compiler.



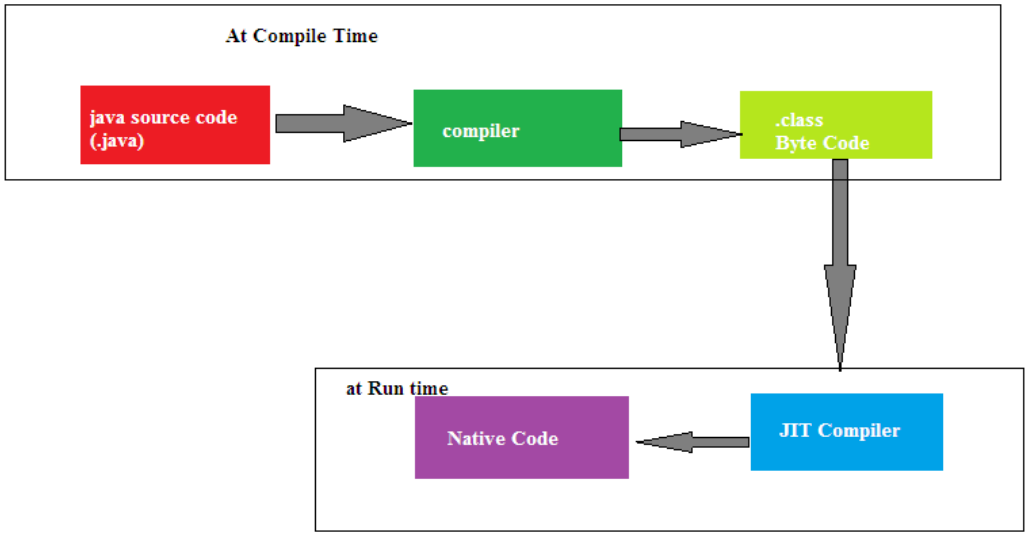


C/C++ Execution

- Process of C code execution



Java Execution



Java Byte Code

- Java source code is compiled to special format called Java Byte Code(.class files).

Bytecode is program code that has been compiled from source code into low-level code designed for a software interpreter.

- Java Byte Code is compiled from Java source code and can be run on a Java Virtual Machine (JVM).
- Java is both interpreted and compiled language.
- This byte code can be run on any machine irrelevant of the architecture. This makes Java Platform independent.
- This makes Java “Write Once Run Anywhere”

Java Byte Code

- Source file (.java) → compiled by javac → bytecode (.class file).
- Bytecode is NOT machine code (not directly executable by hardware) and NOT human-readable source.

It sits in between Consists of a set of instructions (opcodes) designed specifically for the JVM's instruction set like iload, invokevirtual, areturn, etc.

JVM (Java Virtual Machine)

- The runtime engine that actually executes Java bytecode Platform-specific (Windows JVM, Linux JVM, etc.) .

This is what gives Java its "write once, run anywhere" property Includes the class loader, bytecode verifier, and execution engine (interpreter + JIT compiler).

- Does NOT ship standalone. It's bundled inside the JRE

Section Overview

- JRE (Java Runtime Environment)
- = JVM + core libraries (java.lang, java.util, java.io, etc.) + supporting files needed to run Java applications
- Meant for end users who just want to run Java programs, not develop them.
- Does not include compiler (javac), debugger, or other development tools

Section Overview

- JDK (Java Development Kit)
- = JRE + development tools
- Includes: javac (compiler), javadoc, jar, jdb (debugger), jconsole, profilers, etc.
- Meant for developers who need to write and compile Java code

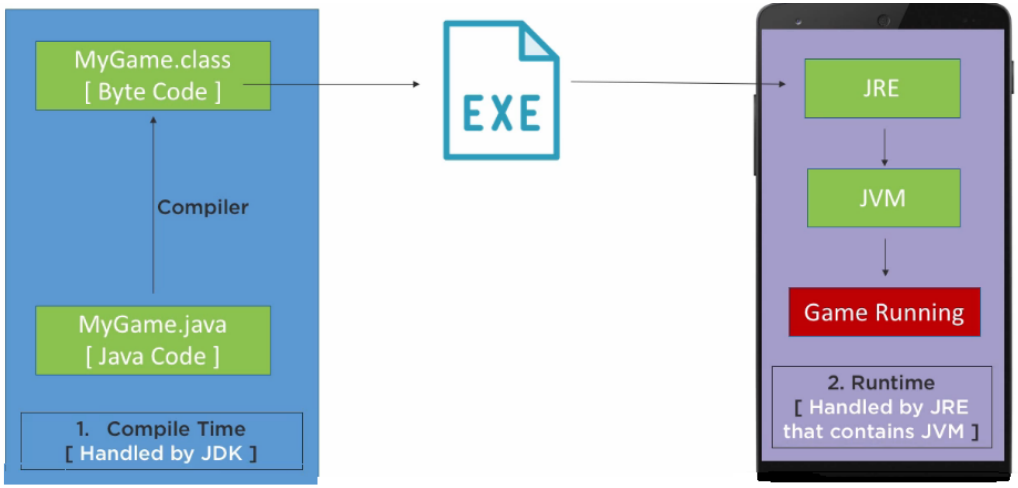
Why Most Other Languages Aren't Platform Independent

- The compiler translates directly into instructions the target CPU understands (x86, ARM, etc.)
- That machine code is tied to:
- The instruction set of the CPU (x86 vs ARM instructions are different)
- The OS's binary format (Windows uses .exe/PE format, Linux uses ELF, macOS uses Mach-O)
- The OS's system calls / APIs the program depends on

Result: a .exe compiled on Windows simply cannot run on Linux, wrong binary format, wrong system calls, possibly wrong CPU instructions too.

Terminologies JDK,JVM,JRE

How Java code runs?



Summary of the Process

- Write: You write the Java source code in .java files.
- Compile: The javac compiler compiles the source code into bytecode (.class files).
- Load: The JVM’s Class Loader loads the bytecode into memory.
- Verify: The Bytecode Verifier checks the bytecode for safety and correctness.

Execute: The JVM executes the bytecode, converting it to native machine code using the JIT compiler for optimized performance.

- Run: The native code is executed on the host machine, running your Java program.
- Manage Memory: The JVM handles memory management, including garbage collection.

FOP Vs OOP

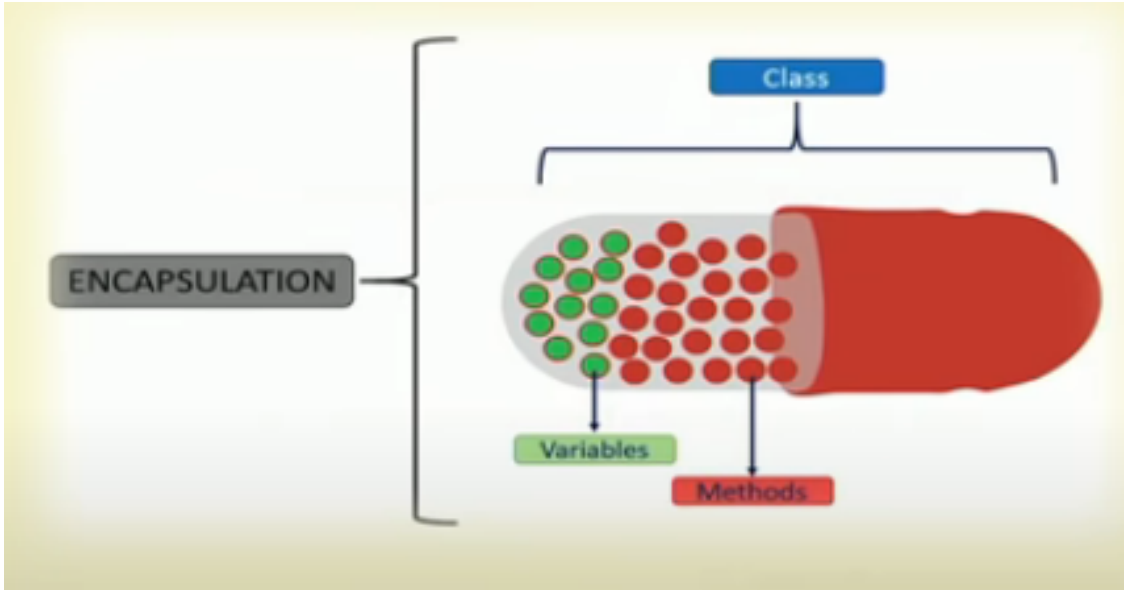
Java programming paradigms

Java is based on the concept of object-oriented programming. Object contain data and functionality to handle data. This is controlled by the following four paradigms.

- Encapsulation
- Inheritance
- Information hiding
- Polymorphism

Encapsulation

Encapsulation is the process of wrapping code and data together into a single unit, for example , capsule which is mixed of several medicines.



Encapsulation Example

- data
- method





Book

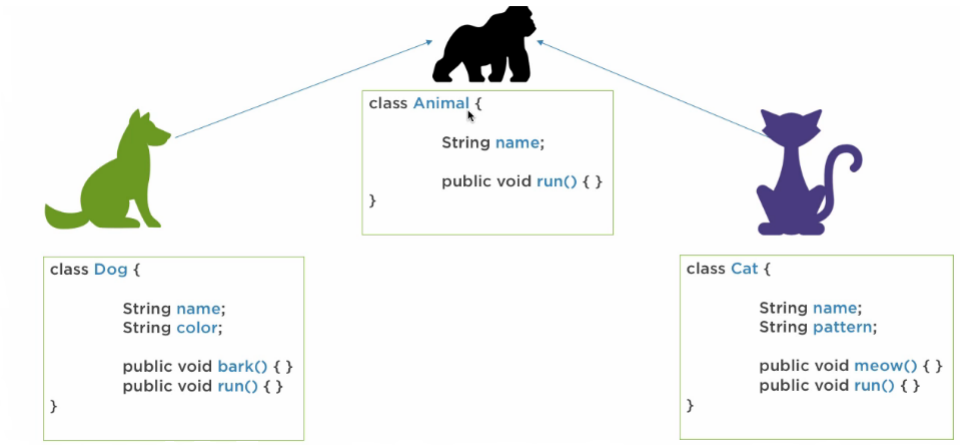
Encapsulation Example

Encapsulation Example

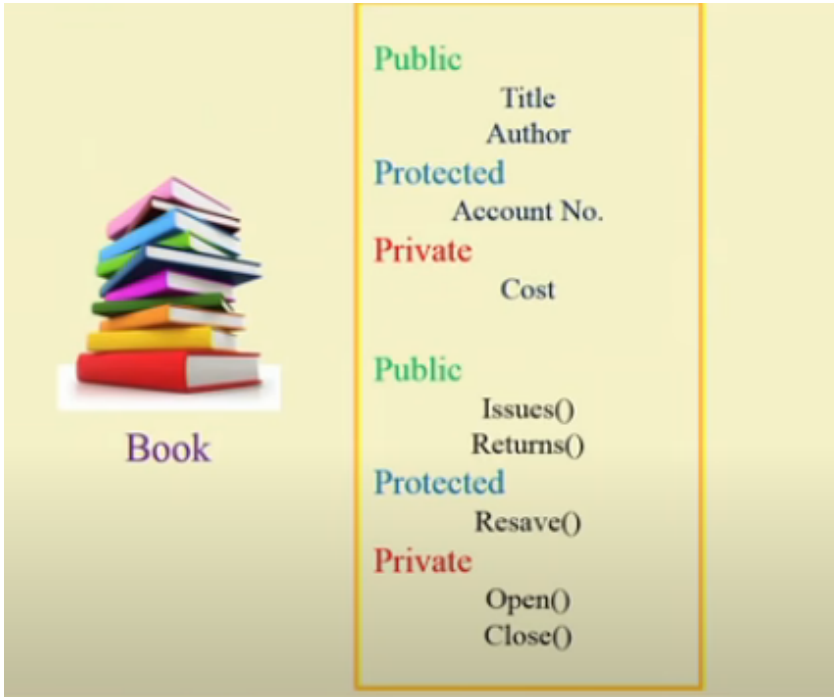
Inheritance

Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important OOPs concept in Java.

Inheritence



Data Hiding


Book
Public
Title
Author
Protected
Account No.
Private
Cost
Public
Issues()
Returns()
Protected
Resave()
Private
Open()
Close()

Polymorphism

Polymorphism in Java is a way where we can perform a single action in many ways. Polymorphism is derived from 2 Greek words: poly and morphs. The word "poly" means many and "morphs" means forms. So polymorphism means many forms.

Polymorphism

First Java Program

- class Example
- {
- /* Sample Program
- Example.java
- */
- public static void main(String[] args)
- {

```
> System.out.println("Hello World");
```

- }
- }

Dissecting a Java Program

- /* …..*/ - For multi line comment.

```
> // - For single line comment
```

main – All java applications begin execution by calling main(). main() is called by java interpreter before any objects are made.

public – an access specifier. main() is declared public which allows the code outside of its class to call main() when the program starts.

- static – allows main() to be called without having to instantiate a particular instance of the class.
- void – tells compiler that main() does not return any value.
- Java is case sensitive.

Executing java program

- First compile the code
  - javac Example.java
- Then execute
  - java Example.java
- File name
- File name

Various IDEs available

- IntelliJ IDEA
- Eclipse
- NetBeans
- VS Code with Java Extensions

Example Program

- Program to add two numbers and display the result

```
> public class AddTwoNumbers {
> public static void main(String[] args) {
> // Declare two integer variables
```

- int num1 = 10;
- int num2 = 20;

```
> // Add the two numbers
```

- int sum = num1 + num2;

```
> // Display the result
> System.out.println("The sum of " + num1 + " and " + num2 + " is: " + sum);
```

- }
- }